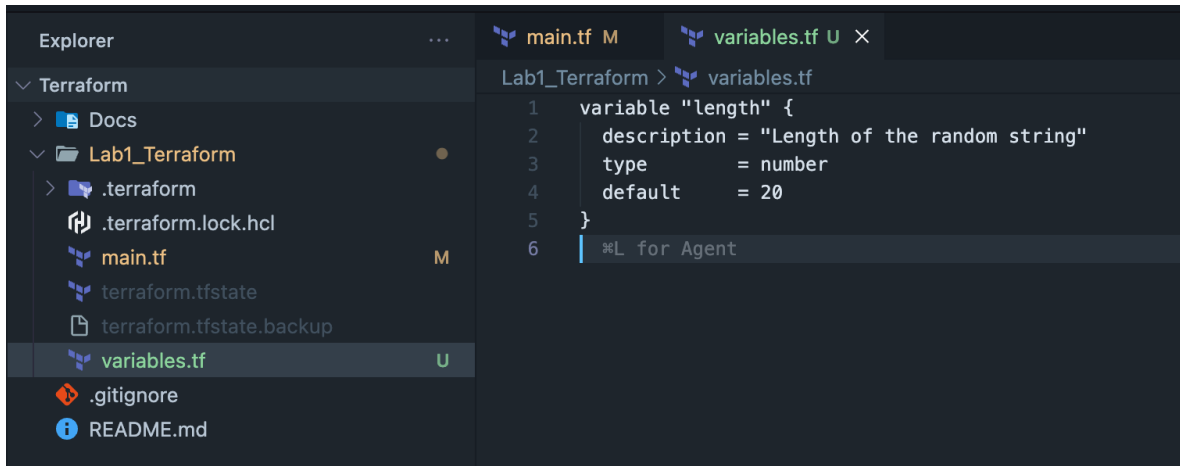


Terraform 2

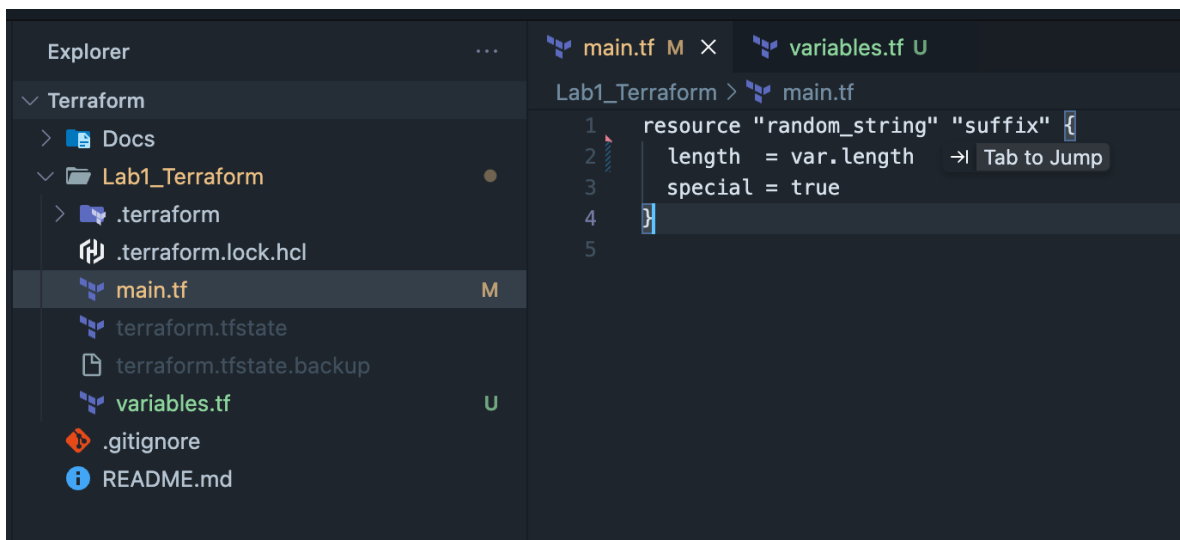
Carlos Daniel Ciriaco Juan – IDGS-103

2.- Variables



This screenshot shows the Visual Studio Code interface with the Terraform project. The Explorer on the left lists the files: Docs, Lab1_Terraform (containing .terraform, .terraform.lock.hcl, main.tf, terraform.tfstate, and terraform.tfstate.backup), .gitignore, and README.md. The main editor displays the variables.tf file with the following content:

```
1 variable "length" {  
2     description = "Length of the random string"  
3     type       = number  
4     default    = 20  
5 }  
6 %L for Agent
```



This screenshot shows the Visual Studio Code interface with the Terraform project. The Explorer on the left lists the files: Docs, Lab1_Terraform (containing .terraform, .terraform.lock.hcl, main.tf, terraform.tfstate, and terraform.tfstate.backup), .gitignore, and README.md. The main editor displays the main.tf file with the following content:

```
1 resource "random_string" "suffix" {  
2     length = var.length  
3     special = true  
4 }  
5
```

```

.../PWA/Terraform/Lab1_Terraform
carlosdc@192 Lab1_Terraform % terraform plan
random_string.suffix: Refreshing state... [id=7zpz]Pe?m(kn7GyF]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
/+ destroy and then create replacement

Terraform will perform the following actions:

  # random_string.suffix must be replaced
  /* resource "random_string" "suffix" {
    ~ id       = "7zpz]Pe?m(kn7GyF" → (known after apply)
    ~ length   = 16 → 20 # forces replacement
    ~ result    = "7zpz]Pe?m(kn7GyF" → (known after apply)
    # (9 unchanged attributes hidden)
  }

Plan: 1 to add, 0 to change, 1 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
carlosdc@192 Lab1_Terraform % terraform apply
random_string.suffix: Refreshing state... [id=7zpz]Pe?m(kn7GyF]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
/+ destroy and then create replacement

Terraform will perform the following actions:

  # random_string.suffix must be replaced
  /* resource "random_string" "suffix" {
    ~ id       = "7zpz]Pe?m(kn7GyF" → (known after apply)
    ~ length   = 16 → 20 # forces replacement
    ~ result    = "7zpz]Pe?m(kn7GyF" → (known after apply)
    # (9 unchanged attributes hidden)
  }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

random_string.suffix: Destroying... [id=7zpz]Pe?m(kn7GyF]
random_string.suffix: Destruction complete after 0s
random_string.suffix: Creating...
random_string.suffix: Creation complete after 0s [id=80PP6U[s$mgcKgnH{o0l]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
carlosdc@192 Lab1_Terraform %

```

3.- Pasar valores a variables

```
carlosdc@192 Lab1_Terraform % terraform plan -var "length=30"
random_string.suffix: Refreshing state... [id=80PP6U[s$mgcKgnH{o0l}]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
/+ destroy and then create replacement

Terraform will perform the following actions:

# random_string.suffix must be replaced
/+ resource "random_string" "suffix" {
  ~ id      = "80PP6U[s$mgcKgnH{o0l}" → (known after apply)
  ~ length  = 20 → 30 # forces replacement
  ~ result  = "80PP6U[s$mgcKgnH{o0l}" → (known after apply)
  # (9 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
carlosdc@192 Lab1_Terraform % terraform apply -var "length=30"
random_string.suffix: Refreshing state... [id=80PP6U[s$mgcKgnH{o0l}]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
/+ destroy and then create replacement

Terraform will perform the following actions:

# random_string.suffix must be replaced
/+ resource "random_string" "suffix" {
  ~ id      = "80PP6U[s$mgcKgnH{o0l}" → (known after apply)
  ~ length  = 20 → 30 # forces replacement
  ~ result  = "80PP6U[s$mgcKgnH{o0l}" → (known after apply)
  # (9 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

random_string.suffix: Destroying... [id=80PP6U[s$mgcKgnH{o0l}]
random_string.suffix: Destruction complete after 0s
random_string.suffix: Creating...
random_string.suffix: Creation complete after 0s [id=ppp%:=X:qr8m!ZgX6#q#bT)fr&v>7r]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
carlosdc@192 Lab1_Terraform %
```

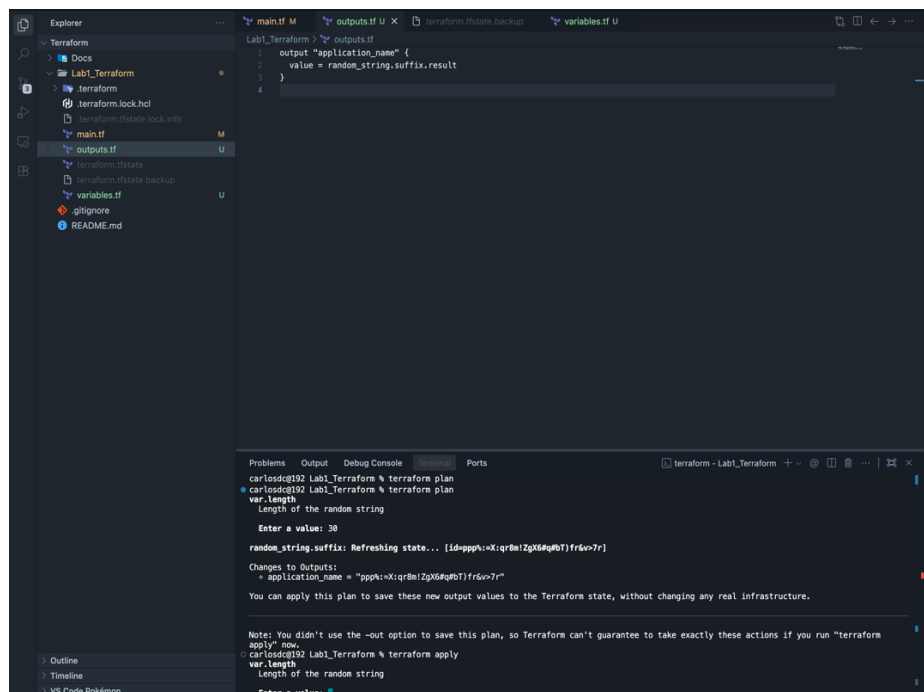
4.- Variables locales



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays a file tree for a project named 'Lab1_Terraform'. The files listed include '.terraform', '.terraform.lock.hcl', 'main.tf' (marked with an 'M'), 'terraform.tfstate', 'terraform.tfstate.backup', 'variables.tf' (marked with a 'U'), '.gitignore', and 'README.md'. The main editor area shows the 'main.tf' file. It contains a Terraform resource definition for 'random_string' with attributes 'length' and 'special'. Below this, a 'locals' block is defined, containing a comment in Spanish and a variable 'unique_name' that concatenates 'var.application_name', 'var.environment', and the result of 'random_string.suffix.result'.

```
1 resource "random_string" "suffix" {
2   length = var.length
3   special = true
4 }
5
6
7 locals {
8   # Genera un nombre único para el recurso utilizando un prefijo y un sufijo aleatorio.
9   unique_name = "${var.application_name}-${var.environment}-${random_string.suffix.result}"
10
11   → Tab to Jump
```

5.- Outputs



The screenshot shows a VS Code editor with a file explorer on the left displaying a project structure for 'Terraform'. The main editor shows the 'outputs.tf' file with the following content:

```
1 output "application_name" {
2   value = random_string.suffix.result
3 }
4
```

The terminal window at the bottom shows the execution of the following commands:

```
carlosdc@192 Lab1_Terraform % terraform plan
carlosdc@192 Lab1_Terraform % terraform plan
var.length
Length of the random string

Enter a value: 30

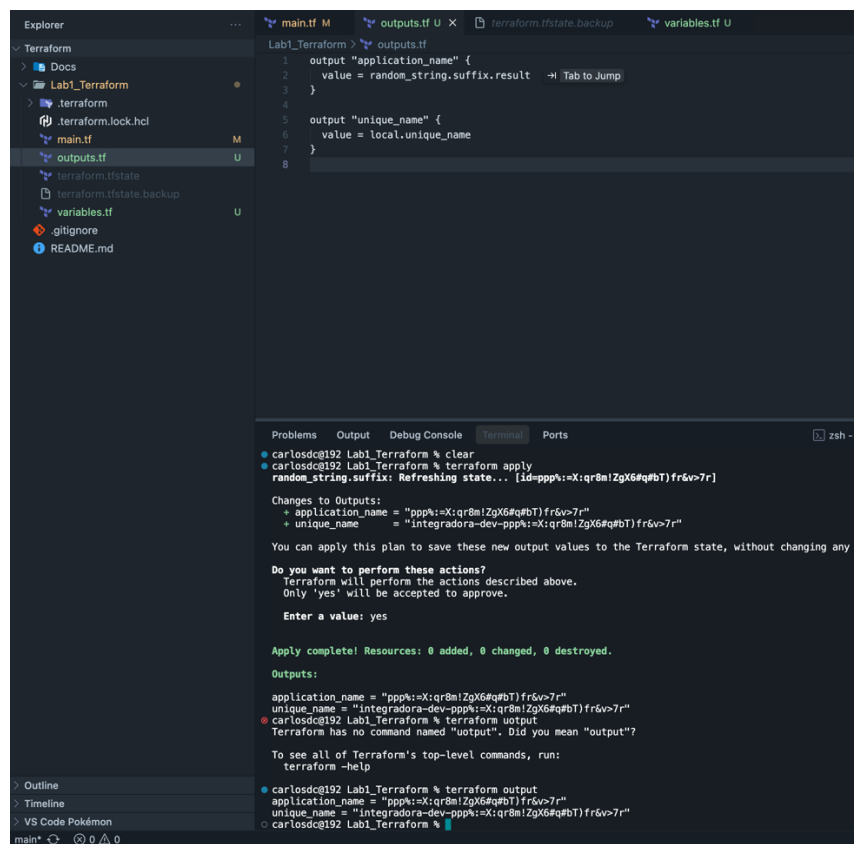
random_string.suffix: Refreshing state... [id=pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r]

Changes to Outputs:
+ application_name = "pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Notes: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
carlosdc@192 Lab1_Terraform % terraform apply
var.length
Length of the random string

Enter a value: 
```



The screenshot shows a VS Code editor with a file explorer on the left displaying a project structure for 'Terraform'. The main editor shows the 'outputs.tf' file with the following content:

```
1 output "application_name" {
2   value = random_string.suffix.result
3 }
4
5 output "unique_name" {
6   value = local.unique_name
7 }
8
```

The terminal window at the bottom shows the execution of the following commands:

```
carlosdc@192 Lab1_Terraform % clear
carlosdc@192 Lab1_Terraform % terraform apply
random_string.suffix: Refreshing state... [id=pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r]

Changes to Outputs:
+ application_name = "pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r"
+ unique_name     = "integradora-dev-pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r"

You can apply this plan to save these new output values to the Terraform state, without changing any

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

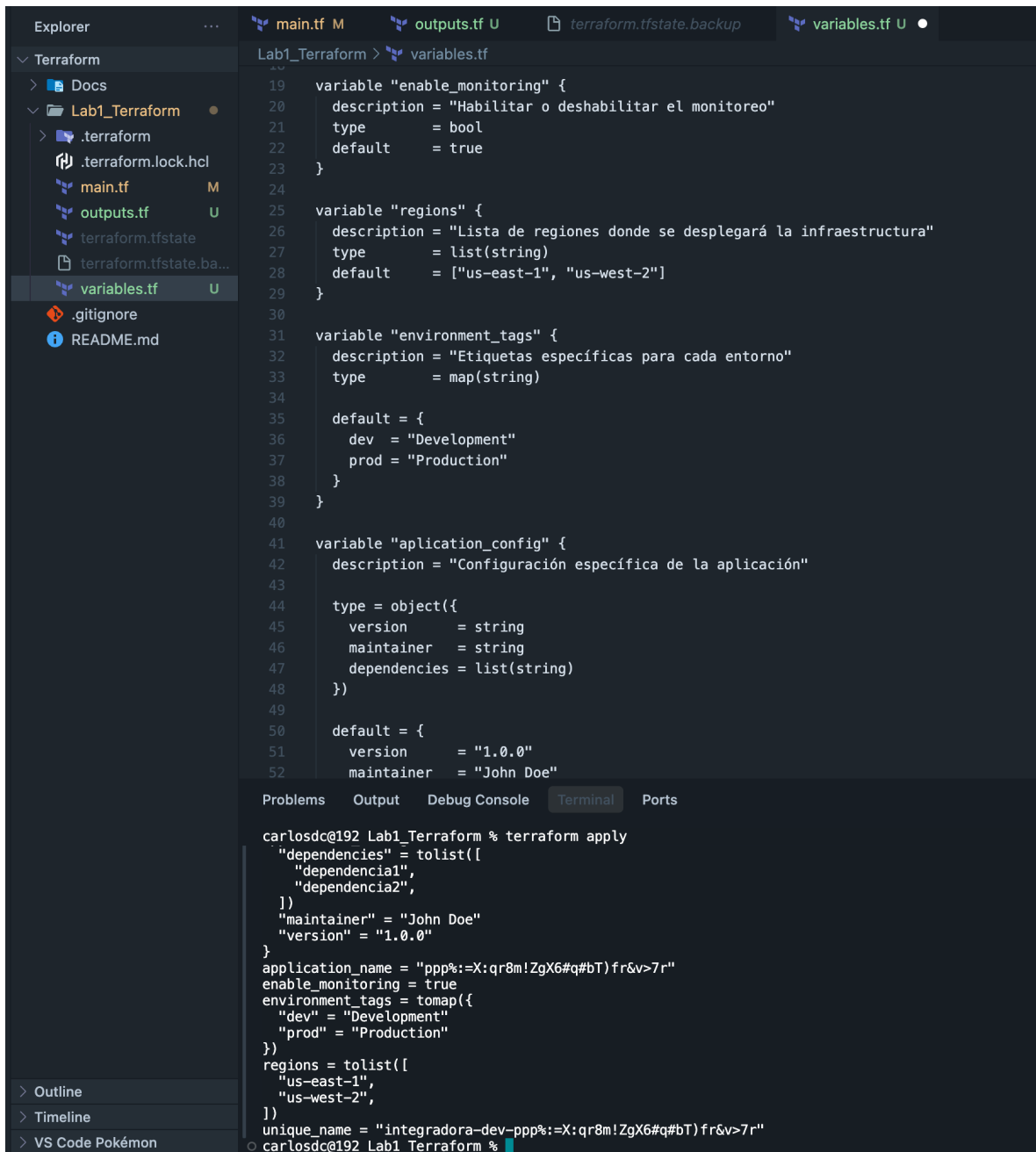
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
application_name = "pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r"
unique_name     = "integradora-dev-pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r"
carlosdc@192 Lab1_Terraform % terraform output
Terraform has no command named "output". Did you mean "output"?

To see all of Terraform's top-level commands, run:
terraform -help

carlosdc@192 Lab1_Terraform % terraform output
application_name = "pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r"
unique_name     = "integradora-dev-pppk:=X:qr8m!ZgX6#q#bT)fr6v>7r"
carlosdc@192 Lab1_Terraform %
```

6.- Tipos de datos en terraform



The image shows a Visual Studio Code editor with a Terraform project. The Explorer sidebar on the left shows the project structure: Terraform > Lab1_Terraform > .terraform, .terraform.lock.hcl, main.tf, outputs.tf, terraform.tfstate, terraform.tfstate.ba..., variables.tf, .gitignore, and README.md. The main editor displays the content of variables.tf, which defines four variables: enable_monitoring (boolean), regions (list of strings), environment_tags (map of strings), and application_config (object). The terminal at the bottom shows the output of the terraform apply command, displaying the values assigned to the variables during the apply process.

```
19 variable "enable_monitoring" {
20   description = "Habilitar o deshabilitar el monitoreo"
21   type       = bool
22   default    = true
23 }
24
25 variable "regions" {
26   description = "Lista de regiones donde se desplegará la infraestructura"
27   type       = list(string)
28   default    = ["us-east-1", "us-west-2"]
29 }
30
31 variable "environment_tags" {
32   description = "Etiquetas específicas para cada entorno"
33   type       = map(string)
34
35   default = {
36     dev = "Development"
37     prod = "Production"
38   }
39 }
40
41 variable "aplication_config" {
42   description = "Configuración específica de la aplicación"
43
44   type = object({
45     version      = string
46     maintainer   = string
47     dependencies = list(string)
48   })
49
50   default = {
51     version      = "1.0.0"
52     maintainer   = "John Doe"
```

```
carlosdc@192 Lab1_Terraform % terraform apply
"dependencies" = tolist([
  "dependencia1",
  "dependencia2",
])
"maintainer" = "John Doe"
"version" = "1.0.0"
}
application_name = "ppp%:=X:qr8m!ZgX6#q#bT)fr&v>7r"
enable_monitoring = true
environment_tags = tomap({
  "dev" = "Development"
  "prod" = "Production"
})
regions = tolist([
  "us-east-1",
  "us-west-2",
])
unique_name = "integradora-dev-ppp%:=X:qr8m!ZgX6#q#bT)fr&v>7r"
carlosdc@192 Lab1_Terraform %
```

7.- Instalación de Azure CLI

```
For more information about Homebrew and Ttyne
● carlosdc@192 Lab1_Terraform % az version
{
  "azure-cli": "2.90.0",
  "azure-cli-core": "2.90.0",
  "azure-cli-telemetry": "1.1.0",
  "extensions": {}
}
○ carlosdc@192 Lab1_Terraform %
```

8.- Conexión usando CLI

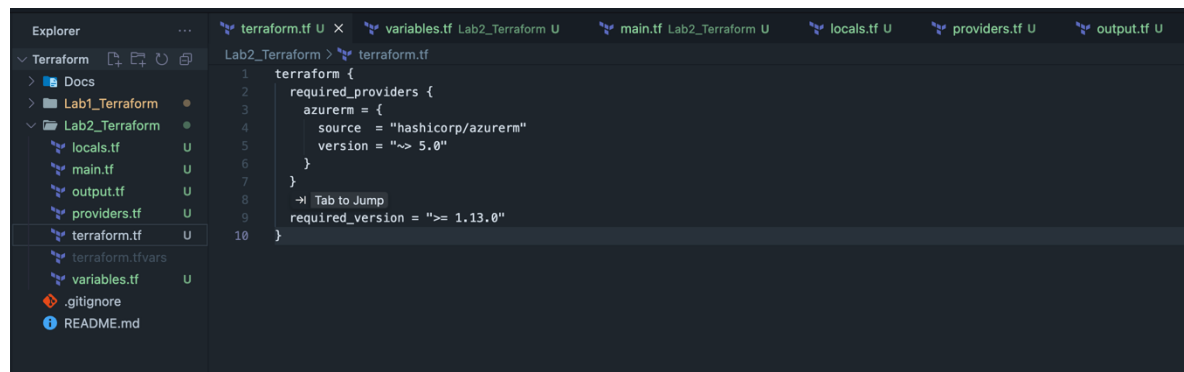
```
Retrieving tenants and subscriptions for the selection...
(Tenant and subscription selection)
No      Subscription name      Subscription ID      Tenant
-----
[1] * Azure for Students  de8edfbf-276b-41f0-829d-ee20551b5138  Universidad Tecnológica del Valle de Toluca
The default is marked with an *; the default tenant is 'Universidad Tecnológica del Valle de Toluca' and subscription is 'Azure for Students' (de8edfbf-276b-41f0-829d-ee20551b5138).
Select a subscription and tenant (Type a number or Enter for no changes):
```

9.- Demostración

10.- Habilitar el proveedor de redes de Microsoft

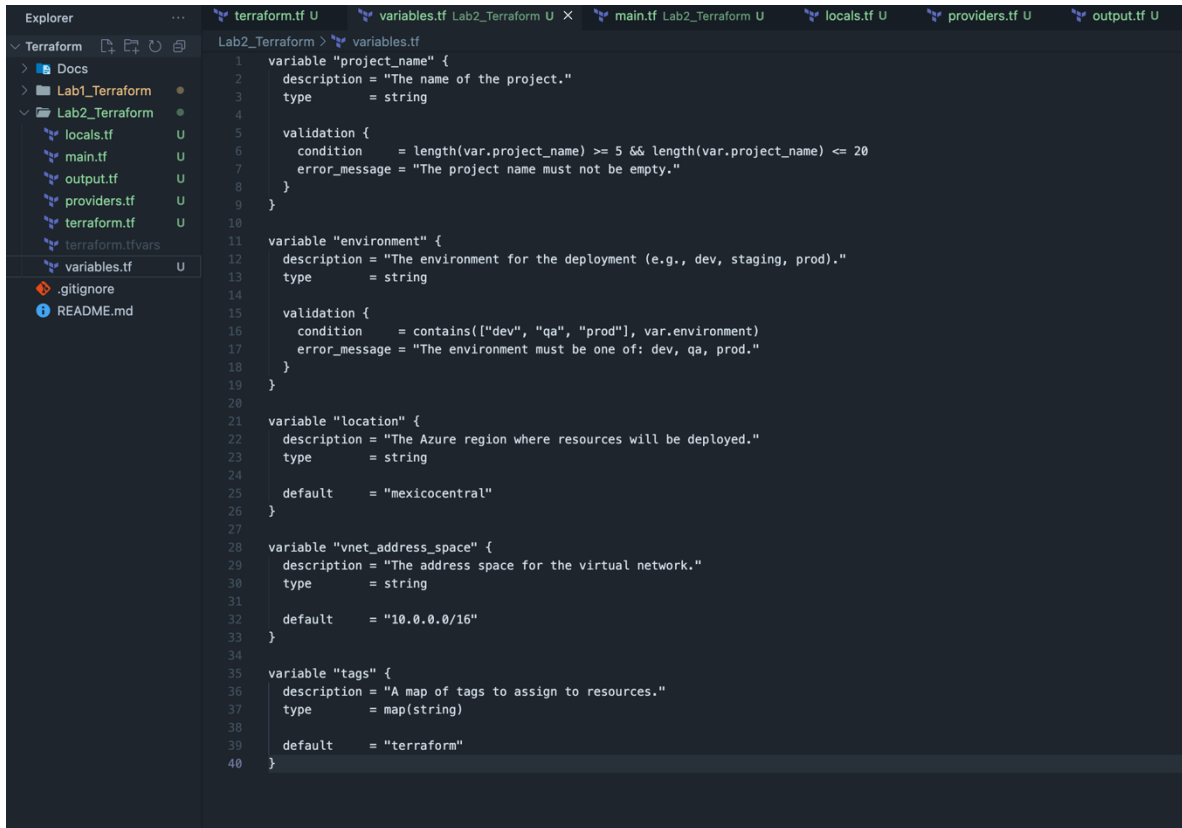
```
● carlosdc@192 Lab2_Terraform % az provider register --namespace Microsoft.Network --subscription de8edfbf-276b-41f0-829d-ee20551b5138 --wait
● carlosdc@192 Lab2_Terraform % az provider show --namespace Microsoft.Network --subscription de8edfbf-276b-41f0-829d-ee20551b5138 --query "{namespace: namespace, state: registrationState}" --output json
{
  "namespace": "Microsoft.Network",
  "state": "Registered"
}
```

11.- Creación de variables y locals



The screenshot shows a VS Code editor with a Terraform project. The Explorer on the left shows a directory structure with files like locals.tf, main.tf, output.tf, providers.tf, terraform.tf, terraform.tfvars, variables.tf, .gitignore, and README.md. The main editor shows the content of terraform.tf, which includes a terraform block with required_providers for azureRM and a required_version constraint.

```
1 terraform {
2   required_providers {
3     azureRM = {
4       source = "hashicorp/azurerm"
5       version = "~> 5.0"
6     }
7   }
8   Tab to Jump
9   required_version = ">= 1.13.0"
10 }
```

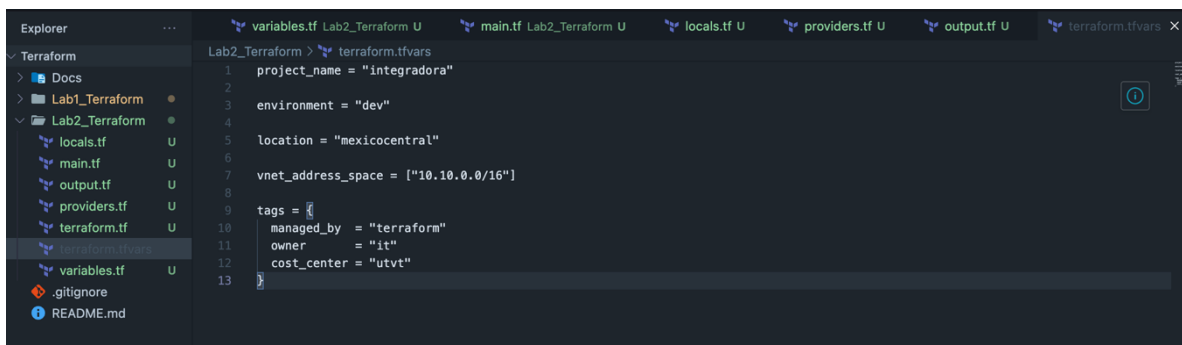


```
1 variable "project_name" {
2   description = "The name of the project."
3   type       = string
4
5   validation {
6     condition = length(var.project_name) >= 5 && length(var.project_name) <= 20
7     error_message = "The project name must not be empty."
8   }
9 }
10
11 variable "environment" {
12   description = "The environment for the deployment (e.g., dev, staging, prod)."
13   type       = string
14
15   validation {
16     condition = contains(["dev", "qa", "prod"], var.environment)
17     error_message = "The environment must be one of: dev, qa, prod."
18   }
19 }
20
21 variable "location" {
22   description = "The Azure region where resources will be deployed."
23   type       = string
24   default    = "mexicocentral"
25 }
26
27 variable "vnet_address_space" {
28   description = "The address space for the virtual network."
29   type       = string
30   default    = "10.0.0.0/16"
31 }
32
33 variable "tags" {
34   description = "A map of tags to assign to resources."
35   type       = map(string)
36   default    = "terraform"
37 }
38
39
40 }
```



```
1 locals {
2   resource_group_name = "rg-${var.project_name}-${var.environment}-${var.location}-001"
3   virtual_network_name = "vnet-${var.project_name}-${var.environment}-${var.location}-001"
4
5   common_tags = merge(
6     var.tags,
7     {
8       "Project"    = var.project_name
9       "Environment" = var.environment
10      "Location"   = var.location
11    }
12   )
13 }
```

12.-Configuracion ftvars y providers

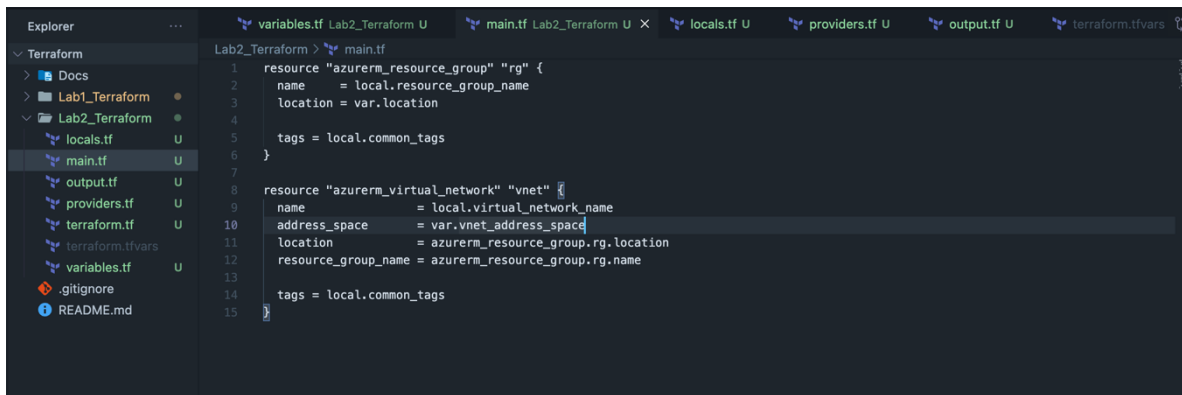


```
1 project_name = "integradora"
2
3 environment = "dev"
4
5 location = "mexicocentral"
6
7 vnet_address_space = ["10.10.0.0/16"]
8
9 tags = {
10   managed_by = "terraform"
11   owner      = "it"
12   cost_center = "utvt"
13 }
```

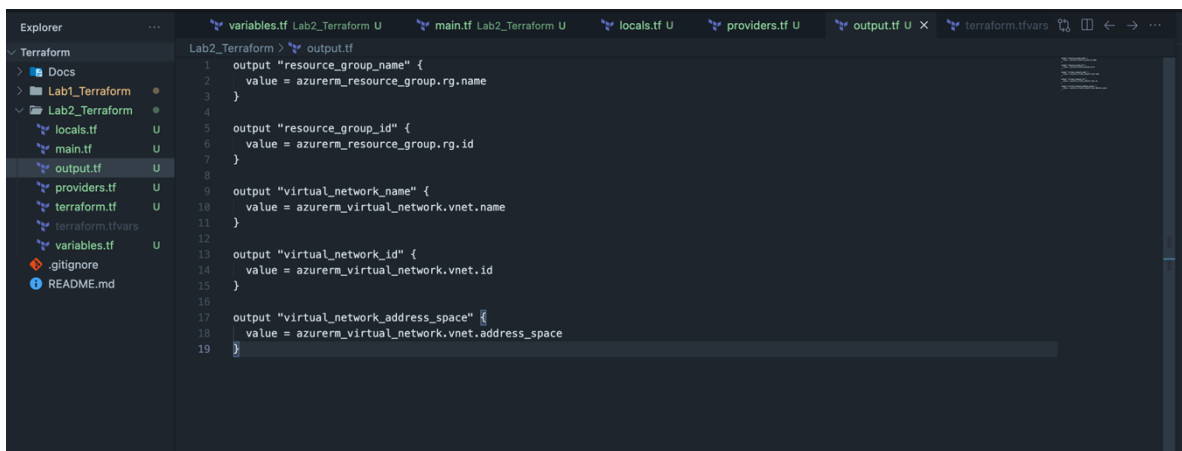
```
variable "subscription_id" {
  description = "The Azure subscription ID."
  type        = string
  default     = "de8edfbf-276b-41f0-829d-ee20551b5138"
}
```



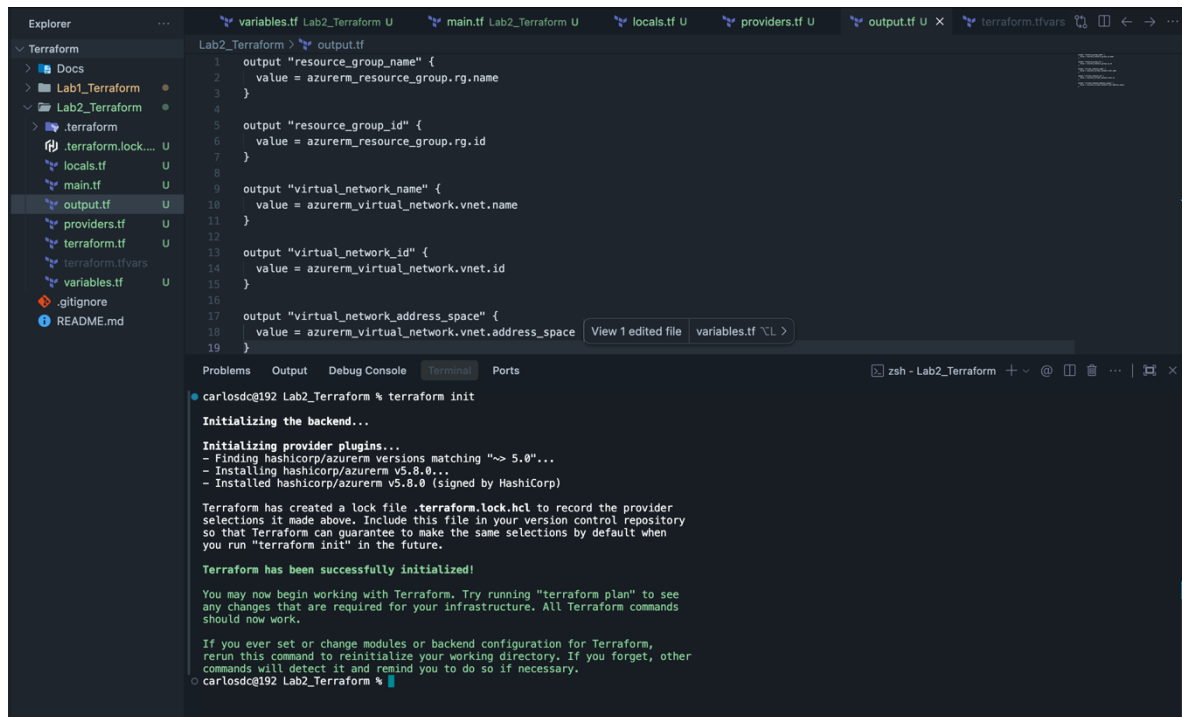
13.- Configuración de recursos



14.-Salidas



15.- Creación de elementos



The screenshot shows a VS Code editor with a file explorer on the left containing Terraform files: Docs, Lab1_Terraform, Lab2_Terraform, .terraform, .terraform.lock.hcl, locals.tf, main.tf, output.tf, providers.tf, terraform.tfvars, variables.tf, .gitignore, and README.md. The main editor displays the content of 'output.tf' with the following HCL code:

```
1 output "resource_group_name" {
2   value = azurem_resource_group.rg.name
3 }
4
5 output "resource_group_id" {
6   value = azurem_resource_group.rg.id
7 }
8
9 output "virtual_network_name" {
10  value = azurem_virtual_network.vnet.name
11 }
12
13 output "virtual_network_id" {
14  value = azurem_virtual_network.vnet.id
15 }
16
17 output "virtual_network_address_space" {
18   value = azurem_virtual_network.vnet.address_space
19 }
```

Below the editor, the terminal window shows the output of the 'terraform init' command:

```
carlosdc@192 Lab2_Terraform % terraform init
Initializing the backend...

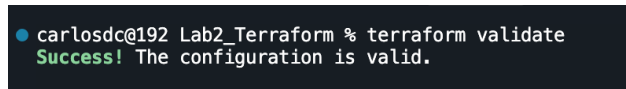
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 5.0"...
- Installing hashicorp/azurerm v5.8.0...
- Installed hashicorp/azurerm v5.8.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

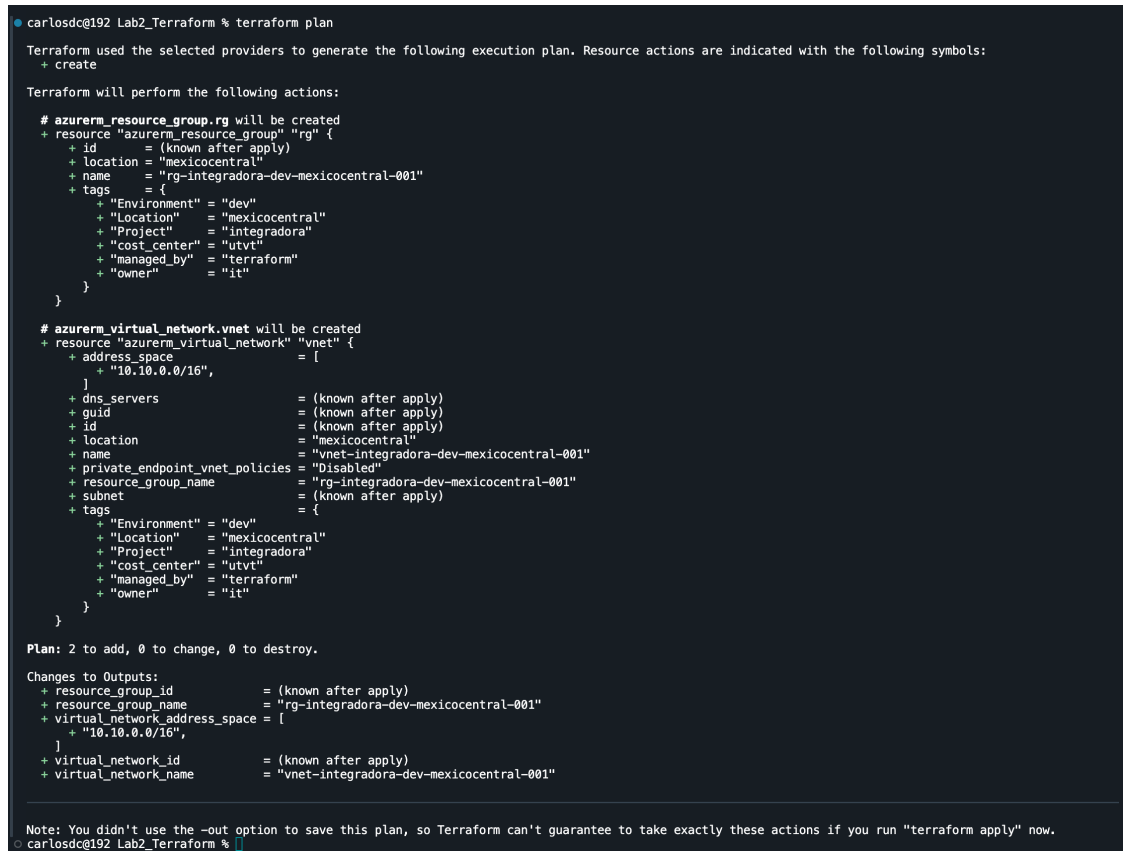
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
carlosdc@192 Lab2_Terraform %
```



```
carlosdc@192 Lab2_Terraform % terraform validate
Success! The configuration is valid.
```



```
carlosdc@192 Lab2_Terraform % terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# azurem_resource_group.rg will be created
+ resource "azurem_resource_group" "rg" {
+   id           = (known after apply)
+   location     = "mexicocentral"
+   name         = "rg-integradora-dev-mexicocentral-001"
+   tags        = {
+     "Environment" = "dev"
+     "Location"    = "mexicocentral"
+     "Project"     = "integradora"
+     "cost_center" = "utvt"
+     "managed_by"  = "terraform"
+     "owner"       = "it"
+   }
+ }

# azurem_virtual_network.vnet will be created
+ resource "azurem_virtual_network" "vnet" {
+   address_space = [
+     "10.10.0.0/16",
+   ]
+   dns_servers   = (known after apply)
+   guid          = (known after apply)
+   id            = (known after apply)
+   location      = "mexicocentral"
+   name          = "vnet-integradora-dev-mexicocentral-001"
+   private_endpoint_vnet_policies = "Disabled"
+   resource_group_name = "rg-integradora-dev-mexicocentral-001"
+   subnet         = (known after apply)
+   tags          = {
+     "Environment" = "dev"
+     "Location"    = "mexicocentral"
+     "Project"     = "integradora"
+     "cost_center" = "utvt"
+     "managed_by"  = "terraform"
+     "owner"       = "it"
+   }
+ }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ resource_group_id           = (known after apply)
+ resource_group_name         = "rg-integradora-dev-mexicocentral-001"
+ virtual_network_address_space = [
+   "10.10.0.0/16",
+ ]
+ virtual_network_id          = (known after apply)
+ virtual_network_name        = "vnet-integradora-dev-mexicocentral-001"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
carlosdc@192 Lab2_Terraform %
```

Microsoft Azure

Buscar recursos, servicios y documentos (G+)

Copilot

carlos.ciriaco@unvtol.edu...
UNIVERSIDAD TECNOLÓGICA DEL...

Inicio > Resource Manager

Resource Manager | Grupos de recursos

Unidad Tecnológica del Valle de Toluca

¿Cómo administrar los cambios con las herramientas de implementación? Resumir mis costos por servicio Exportar grupos de recursos con Bicep o Terraform

Buscar

+ Crear Administrar vista Actualizar Exportar a CSV Abrir consulta Asignar etiquetas Analizar selección Agregar a un grupo de servicios No agrupar

Filtrar por cualquier...

Suscripción es igual que todo Ubicación es igual que todo Agregar filtro

Resource Manager

Todos los recursos

Recursos favoritos

Recursos recientes

Grupos de recursos

Etiquetas

Organización

Herramientas

Implementaciones

Help

No hay grupos de recursos para mostrar

Los grupos de recursos proporcionan un contenedor lógico para administrar y organizar los recursos de Azure, lo que simplifica la administración y permite una administración eficaz de los recursos.

+ Crear

Más información

Agregue o quite favoritos presionando Ctrl+Shift+F

Mostrando 1 - 0 de 0. Recuento de pantallas: auto

Nueva experiencia de exploración Enviar comentarios

Microsoft Azure

Buscar recursos, servicios y documentos (G+)

Copilot

carlos.ciriaco@unvtol.edu...
UNIVERSIDAD TECNOLÓGICA DEL...

Inicio > Resource Manager

Resource Manager | Todos los recursos

Unidad Tecnológica del Valle de Toluca

¿Afecta cualquier recurso a las interrupciones? Examinar todos los recursos de esta suscripción Filtrar recursos por región

Buscar

+ Crear Administrar vista Actualizar Exportar a CSV Abrir consulta Asignar etiquetas Eliminar Analizar selección Agregar a un grupo de servicios No agrupar

Filtrar por cualquier...

Suscripción es igual que todo Grupo de recursos es igual que todo Tipo es igual que todo Ubicación es igual que todo Agregar filtro

Resource Manager

Todos los recursos

Recursos favoritos

Recursos recientes

Grupos de recursos

Etiquetas

Organización

Herramientas

Implementaciones

Help

No hay recursos que coincidan con los filtros

Intente cambiar o borrar los filtros.

+ Crear Borrar filtros

Más información

Agregue o quite favoritos presionando Ctrl+Shift+F

Mostrando 1 - 0 de 0. Recuento de pantallas: auto

Nueva experiencia de exploración Enviar comentarios

Inicio > Resource Manager

Resource Manager | Grupos de recursos

Resumir mis costos por servicio

Exportar grupos de recursos con Bicep o Terraform

¿Cómo administrar los cambios con las herramientas de implementación?

Universidad Tecnológica del Valle de Toluca

Buscar

+ Crear | Administrar vista | Actualizar | Exportar a CSV | Abrir consulta | Asignar etiquetas | Analizar selección | Agregar a un grupo de servicios | No agrupar

Resource Manager

Todos los recursos

Recursos favoritos

Recursos recientes

Grupos de recursos

Etiquetas

> Organización

> Herramientas

> Implementaciones

> Help

Filtrar por cualquier ...

Suscripción es igual que todo

Ubicación es igual que todo

+ Agregar filtro

☐	Nombre ↑	Suscripción	Ubicación
☐	NetworkWatcherRG	... Azure for Students	Mexico Central
☐	rg-integradora-dev-mexicocentral-001	... Azure for Students	Mexico Central